

pavan_ca1_thesis (1).pdf

 National Law University, Odisha

Document Details

Submission ID

trn:oid:::29834:128168862

Submission Date

Feb 13, 2026, 12:33 PM GMT+5:30

Download Date

Feb 13, 2026, 12:35 PM GMT+5:30

File Name

pavan_ca1_thesis (1).pdf

File Size

122.8 KB

3 Pages

1,345 Words

8,167 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **0 AI-generated only 0%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Multi-Tenant SaaS Identity and Quota Management: A Unified Framework for Tenant-Aware Authentication, Isolation Enforcement, and Fair Resource Allocation

Pavan Sai Bodduluru
X24211371
Research in Computing CA1
National College of Ireland

1 Research Problem Background

As Software-as-a-Service (SaaS) continues to dominate the Cloud delivery model, the advent of multi-tenancy has allowed for the resource-sharing phenomenon between tenants while still maintaining their data isolation Pushpan (2024). Nonetheless, tenants continue to experience three critical ongoing challenges: first, insufficient tenant isolation has resulted in the ability for data to be leaked between tenants and create unauthorised access to tenants' data Solanki et al. (2016); second, the inconsistent enforcement of access control has allowed tenants to access resources they do not have permission to utilise when the application layer has not propagated tenant context; and third, the absence of quota management has created the ongoing problem of "noisy neighbours," where tenants that utilise, and are therefore resource intensive, will degrade the performance of their co-located tenants Cheng et al. (2025), Walraven et al. (2015).

Existing solutions for identity, isolation, and quota management have been created as independent concerns resulting in an architecture characterised by fragmentation Krebs et al. (2012). Furthermore, the identity services used do not embed the tenant context within the identity token, thus requiring expensive lookups against the tenant's identity in the database for every request made against their resources. There are multiple approaches for isolating databases, from shared schema with row-level security to database-per-tenants. Each approach provides dissimilar scaling advantages and costs for the tenants Yaish & Goyal (2013). Likewise, Quota systems are usually enforce API rate limits only, while foregoing to monitor other resource types consumed relative to storage, compute, and network. Consequently, an integrated framework has not yet unified these components in a common architectural model with tenant awareness Sandhu et al. (1996).

2 Research Question

How can a unified tenant-aware framework integrate authentication mechanisms (JWT tokens with tenant_id claims), application-layer enforcement (middleware-based query filtering), database-level security (row-level policies), and multi-dimensional quota management (API calls, storage, compute) to achieve measurable improvements in cross-tenant security (zero unauthorized access), resource fairness (Jain's index > 0.9), and performance (authentication $< 50\text{ms}$, quota overhead $< 5\text{ms}$) in multi-tenant SaaS environments?

3 Justification

Significance: This solution provides a significant resolution to the serious gap impacting 80% of organizations transitioning to a Software as a Service (SaaS) computing environment. Security breaches associated with improper isolations cause \$4,460,000 per incident (IBM 2020). Developing an integrated framework can reduce the complexity associated with environments, improve their overall security, and assist with the equitable allocation of resources.

Feasibility: Using existing technologies for authentication (JWT/OAuth2), database isolation (using rearranging security either PostgreSQL row-level security or MongoDB filtering), and tracking quotas (using Redis atomic counters), this project can be completed within 12 weeks. The actual implementation consists of using Spring Security/Passport.js middlewares in combination with either a Python FastAPI or Node.js Express backend, along with a React-based administrative interface. All of these tools are open-source and documented.

Measurability: Quantified through: (1) Security: zero cross-tenant breaches in 10,000 penetration attempts; (2) Performance: authentication latency < 50ms, quota checks < 5ms, throughput of 10,000 concurrent requests across 100 tenants; (3) Fairness: Jain's index > 0.9 for resource distribution. Evaluation uses YCSB database benchmarks and custom multi-tenant workload generators simulating realistic access patterns with varying tenant sizes.

Ethics: Use only artificial tenant information and de-identified access patterns to strengthen privacy by enhancing the ability to isolate these types of systems from other systems, therefore, fulfilling the criteria for both GDPR Article 32 HIPAA Security Rule. There are no human subjects nor actual client data used in this process.

4 Specific Items to Be Addressed

1. **Tenant-Aware Authentication:** Establish a centralised service that will create JWT Tokens that contain either a tenant_id or role claim. The service will also support local credentials as well as SAML 2.0 federation and OpenID Connect. The service will store configurable IdP settings in regard to the tenants which will be protected through encryption. The service will also have the capability of refreshing and revoking tokens.

Measurements: Authentication throughput (request/second), Back-end validation latency (ms), Federation overhead (%).

2. **Request-Level Enforcement:** Develop middleware extracting tenant context from tokens and enforcing mandatory filtering on database queries through ORM interceptors (Hibernate, Sequelize, SQLAlchemy). Include audit logging and anomaly detection for cross-tenant access attempts. *Metrics:* Query filtering overhead (%), false positive rate (%), concurrent capacity (requests/sec).

3. **Database Isolation:** The three strategies under study will be implemented and evaluated in the same manner. The strategies that will be used for implementation and evaluation of the strategies will be a combination of; (1) shared schema with row-level security, (2) tenant schema and routing, and (3) a tenant database using connection pooling. To assist with the evaluation of the three strategies, the study will create automated tools to migrate or create tenant discriminators that have the proper indexing. The metrics that will be used to evaluate the three strategies are query overhead by number of tenants, (10, 50, 100 500), isolation effectiveness, and scalability limits.

4. **Quota Management:** Create an application programming interface (API) that will track the consumption of resources that are being used by services that are being provided. To avoid race conditions, use atomic operations (i.e., the WATCH, MULTI and EXEC commands) to enforce these restrictions. Your application must be capable of rejecting requests that exceed a hard limit and alert you when consuming in excess of a soft limit; you also must allow for usage to exceed the amount of available quotas (bursting). Your API must expose resource consumption data through RESTful API calls and webhooks.

Performance Measurement: Time from when a consumer requests resource to when resource consumption request is sent to consumer; Amount of Time Request will be Queued While Waiting for Quota Check at Various Concurrency Levels; Accuracy of Race Condition Detection/Prevention; Enforcement of Hard Limits and Soft Limits (%); Time Required to Provide Consistency Guarantees.

5. **Admin Interface:** Creating a dashboard that provides CRUD capabilities to manage new users providing access to the mobile application, users who will be on boarded, suspended users and deleted

users. In addition, the dashboard will provide Identity Provider Configuration, quota template allocation to tenants and tenant quotas with time series charts and heat maps for usage.

Metrics: task completion time, number of errors, and System Usability Scale score (≥ 80).

- 6. Security Validation:** A penetration test s/or encompasses: Manipulation of tenant ids, Forged token generation, Circumventing quota enforcement and SQL injection attacks. The performance of E2E Latency; DB Overhead; Authentication/Quota subsystems will be measured/relative to three (3) baseline configurations; Single tenant, App Only Filtered and DB Only Isolated.

Deliverables: Security evaluation report, performance latency plots (p50/p95/p99), comparative overhead evaluation.

References

- Cheng, A., Agrawal, P., Cheung, A. et al. (2025), 'Fair transaction processing for multi-tenant databases', *Proceedings of the VLDB Endowment* **18**(10), 2602–2615.
- Krebs, R., Momm, C. & Kounev, S. (2012), Architectural concerns in multi-tenant saas applications, in 'Proceedings of the 2nd International Conference on Cloud Computing and Services Science', pp. 426–431.
- Pushpan, S. (2024), 'Multi-tenant architecture: A comprehensive framework for building scalable saas applications', *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* **10**(6), 1117–1126.
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L. & Youman, C. E. (1996), 'Role-based access control models', *IEEE Computer* **29**(2), 38–47.
- Solanki, N., Zhu, W., Yen, I.-L., Bastani, F. & Rezvani, E. (2016), Multi-tenant access and information flow control for saas, in '2016 IEEE International Conference on Web Services (ICWS)', IEEE, pp. 99–106.
- Walraven, S., De Borger, W., Vanbrabant, B., Decat, M., Lagaisse, B. & Joosen, W. (2015), Adaptive performance isolation middleware for multi-tenant saas, in '2015 IEEE/ACM 8th International Conference on Utility and Cloud Computing (UCC)', IEEE, pp. 394–399.
- Yaish, H. & Goyal, M. (2013), Multi-tenant database access control, in '2013 International Conference on Computational Science and Engineering', IEEE, pp. 870–877.